

```
# 🚀 AGENT_SOP.md — Standard Operating Protocol for AI Agents
### Applies to: Antigravity IDE · Gemini CLI · Claude Code · Cursor · Codex CLI ·
OpenCode · AdaL CLI
### Version: 3.0.0 — FINAL | Maintainer: RapidXAI | February 2026
### Source Skills: Antigravity Awesome Skills (868+ skills) —
github.com/sickn33/antigravity-awesome-skills

---

> ⚠️ **ABSOLUTE MANDATE:** Every AI agent — regardless of tool, model, or task — MUST
read and execute this SOP in full before writing a single line of code, generating a
single component, or making any architectural decision. This document IS the operating
system of every project. No exceptions. No shortcuts.

---

## 🧠 CORE PHILOSOPHY: THE TANDEM SKILL DOCTRINE

> Read this before everything else. This is the single most important concept in this
document.

**A single skill is a single expert. Every real project needs a full team.**

You are NOT limited to invoking one skill at a time. Invoking a single skill for most
tasks is negligence. You MUST evaluate every task, identify every discipline it
touches, and invoke ALL relevant skills simultaneously or in a defined sequence.

### The Three Laws of Skill Invocation

**Law 1 — Never use one skill where two or more apply.**
If a task touches design AND code, invoke both `@ui-ux-pro-max` AND `@react-patterns`.
If it touches code AND security, invoke both `@typescript-expert` AND
`@api-security-best-practices`. Always.

**Law 2 — Skills within the same layer run in parallel. Skills across layers run in
sequence.**
You can invoke `@frontend-design` + `@react-patterns` + `@tailwind-patterns` at the
same time because they are all frontend skills operating on the same surface. But you
run planning BEFORE building, and testing AFTER building.

**Law 3 — Declare your full skill stack before you touch anything.**
```

At the start of every session and every major task, output a skill stack declaration (see Phase 0). An undeclared skill is an unused skill.

The Tandem Mental Model

Think of every project as a building with floors. Each floor needs its own specialist team, and no floor can be skipped:

...

PROJECT STACK - ALL FLOORS MUST BE STAFFED

```
|— Floor 6: Business      → @brainstorming + @concise-planning + @copywriting
|— Floor 5: Design        → @ui-ux-pro-max + @frontend-design + @tailwind-patterns
|— Floor 4: Frontend      → @react-best-practices + @react-patterns +
@nextjs-best-practices
|— Floor 3: Backend       → @senior-fullstack + @backend-dev-guidelines +
@api-patterns
|— Floor 2: Data          → @database-design + @postgres-best-practices + @sql-pro
|— Floor 1: Security      → @api-security-best-practices + @backend-security-coder +
@frontend-security-coder
|— Floor 0: Infra         → @docker-expert + @deployment-procedures +
@observability-engineer
```

...

You do not need every floor for every task. But you MUST identify which floors are touched and staff them all.

Tandem Invocation - Real Syntax Examples

Invoke multiple skills in a single prompt, explicitly:

...

"Use @ui-ux-pro-max and @react-patterns together to build the hero section.
Also apply @tailwind-patterns for all styling and @seo-audit for meta structure."

...

...

"Invoke @senior-fullstack alongside @backend-dev-guidelines and @api-patterns
to design and implement the authentication module, with @api-security-best-practices
and @auth-implementation-patterns running in parallel for every decision."

...

...

```
"Run @test-driven-development, @e2e-testing-patterns, and @browser-automation
in tandem to produce unit, integration, and E2E tests for the checkout flow."
...
```

```
...

"Use @brainstorming and @concise-planning together to plan the sprint,
then use @product-manager-toolkit to produce the PRD."
...
```

📄 TABLE OF CONTENTS

1. [Phase 0 – Pre-Boot & Skill Stack Declaration] ([#phase-0](#))
2. [Phase 1 – Project Discovery & Intelligence Gathering] ([#phase-1](#))
3. [Phase 2 – Architecture & System Design] ([#phase-2](#))
4. [Phase 3 – Web Design & UI/UX Execution] ([#phase-3](#))
5. [Phase 4 – Senior Code Development] ([#phase-4](#))
6. [Phase 5 – Senior Code Analysis & Review] ([#phase-5](#))
7. [Phase 6 – Security Hardening] ([#phase-6](#))
8. [Phase 7 – Infrastructure & Deployment] ([#phase-7](#))
9. [Phase 8 – Testing & QA] ([#phase-8](#))
10. [Phase 9 – Documentation & Handoff] ([#phase-9](#))
11. [Phase 10 – Post-Deployment & Observability] ([#phase-10](#))
12. [Master Tandem Playbook – Pre-Built Combos by Project Type] ([#tandem-playbook](#))
13. [Full Skill Reference by Bundle] ([#skill-reference](#))
14. [Emergency Protocols] ([#emergency](#))
15. [Project Brain Management] ([#brain](#))
16. [Session Completion Checklist] ([#checklist](#))

PHASE 0 – PRE-BOOT & SKILL STACK DECLARATION {#phase-0}

> ****Trigger:**** ALWAYS. The very first thing in every session, before any other action.

0.1 – Read the Brain Directory First

```
```bash
cat .gemini/antigravity/brain/architecture.md 2>/dev/null
cat .gemini/antigravity/brain/decisions.md 2>/dev/null
cat .gemini/antigravity/brain/stack.md 2>/dev/null
```

```
cat .gemini/antigravity/brain/skill-tandems.md 2>/dev/null
cat .gemini/antigravity/brain/conventions.md 2>/dev/null
...
```

If brain files exist → load ALL prior decisions and do not contradict them unless explicitly told to.

### ### 0.2 – Verify Skill Installation

```
```bash
# Check universal path first
test -d ~/.agent/skills && echo "✅ Skills ready at ~/.agent/skills"

# If not found, install immediately:
npx antigravity-awesome-skills

# Tool-specific fallbacks:
# npx antigravity-awesome-skills --claude → .claude/skills/
# npx antigravity-awesome-skills --gemini → .gemini/skills/
# npx antigravity-awesome-skills --cursor → .cursor/skills/
# npx antigravity-awesome-skills --codex → .codex/skills/

# If 404 error:
# npx github:sickn33/antigravity-awesome-skills
...

```

0.3 – Mandatory Skill Stack Declaration

Output this block BEFORE any work begins. It is non-negotiable:

...

AGENT ROLE:	[Full-Stack Dev / Architect / Designer / All]
PROJECT:	[Project Name]
TECH STACK:	[Next.js / Python / N8N / Supabase / etc.]
SESSION GOAL:	[Precise description of what we build today]
ACTIVE SKILL STACK (TANDEM GROUPS):	
└ Planning:	@brainstorming + @concise-planning
└ Design:	@ui-ux-pro-max + @frontend-design
└ Frontend:	@react-patterns + @tailwind-patterns
└ Backend:	@senior-fullstack + @api-patterns

```

|| └─ Security:    @api-security-best-practices + @backend-security-coder ||
|| └─ Testing:    @test-driven-development + @e2e-testing-patterns ||
|| └─ Deploy:     @deployment-procedures + @docker-expert ||
|| ||
|| RULE: Minimum 3 skills active. Minimum 2 invoked in tandem ||
||         for any non-trivial task. ||
||=====||

```

...

0.4 – Establish Operating Context

Before writing anything, answer these internally:

...

- What is the business goal? (Not just the technical ask)
- Which floors of the project stack does this session touch?
- What constraints exist? (Budget, timeline, API limits, hosting)
- Who is the end user? (B2B / B2C / Developer / Non-technical)
- What integrations are required?

...

PHASE 1 – PROJECT DISCOVERY & INTELLIGENCE GATHERING {#phase-1}

> ****Trigger:**** New project, new major feature, or first session on an unfamiliar codebase.

1.1 – Codebase Scan

```

```bash
find . -type f \(-name "*.ts" -o -name "*.tsx" -o -name "*.js" -o -name "*.py" \) |
head -80
cat package.json 2>/dev/null
cat requirements.txt 2>/dev/null
cat .env.example 2>/dev/null
ls -la
...

```

### ### 1.2 – Planning Tandem (Always These Three Together)

...

```
Use @brainstorming and @concise-planning together to analyze the project goal
and produce a prioritized, structured development plan.
Simultaneously use @product-manager-toolkit to define acceptance criteria
and RICE-score the feature priorities.
...
```

```
These three always run together. `@brainstorming` generates ideas →
`@concise-planning` structures them → `@product-manager-toolkit` turns them into a
shippable spec.
```

### ### 1.3 – Business Context Alignment

Resolve all of these before writing a single line of code:

- What is the core user journey from landing to conversion?
- What does "done" look like for this milestone?
- What are the hard constraints? (deadline, spend cap, team size)
- What existing systems must this connect to?

### ### 1.4 – Preliminary Risk Scan (Dual-Skill)

...

```
Use @vulnerability-scanner and @api-security-best-practices together
to perform a preliminary dual-layer risk scan on the project type
and any third-party integrations involved.
...
```

---

## ## PHASE 2 – ARCHITECTURE & SYSTEM DESIGN {#phase-2}

```
> **Trigger:** New project, major refactor, or any feature touching more than 3 files.
```

### ### 2.1 – Architecture Triple Invocation

...

```
Use @senior-architect, @architecture-patterns, and @architecture-decision-records
together:
```

- @senior-architect: owns high-level system design, scaling strategy, component boundaries
- @architecture-patterns: selects the right pattern (Clean Architecture, DDD, Hexagonal, Microservices)

```
- @architecture-decision-records: produces a formal, dated ADR for every major decision

All three run in tandem - the architect's decisions feed directly into the pattern selection,

which feeds directly into the ADR document.

...
```

### ### 2.2 - Stack Selection Criteria (Non-Negotiable)

```
| Criterion | Minimum Standard |
|---|---|
| Scalability | Handles 10x current load without architecture change |
| Maintainability | Any developer understands the codebase in <30 min |
| Cost | Monthly infra cost documented before first commit |
| Security | Auth, RBAC, and data isolation designed at layer 0 |
| Developer Experience | Hot reload, strict typing, linting active from day 1 |
```

### ### 2.3 - API Contract Design (Tandem)

```
...

Use @senior-fullstack, @api-patterns, and @typescript-expert together.
@api-patterns selects the API style (REST vs GraphQL vs tRPC).
@senior-fullstack designs the contract shape and data flow.
@typescript-expert defines strict TypeScript types and interfaces.
If Python: swap @typescript-expert for @python-patterns or @fastapi-pro.

...
```

### ### 2.4 - Database Schema Design

```
...

Use @database-design and @postgres-best-practices together for all schema work.
@database-design: normalization, relationships, ORM selection, indexing strategy.
@postgres-best-practices: Postgres-specific optimization, query patterns, RLS policies.

For Supabase projects: also invoke @supabase (if available) for native Supabase patterns.

...
```

### ### 2.5 - Write Architecture to Brain

After this phase, IMMEDIATELY write all decisions:

```

```bash
mkdir -p .gemini/antigravity/brain/
# Write: architecture summary, stack decisions, ADRs, API contracts to brain files
```

PHASE 3 – WEB DESIGN & UI/UX EXECUTION {#phase-3}

> **Trigger:** Any frontend work – landing pages, dashboards, SaaS UIs, client websites.

3.1 – The Designer + Developer Core Duo (MANDATORY PAIRING)

> This is the most fundamental tandem in all frontend work. These two skills are NEVER separated.

...

Use @ui-ux-pro-max and @react-patterns together for ALL frontend work.
This pairing is non-negotiable. One without the other produces incomplete output.

@ui-ux-pro-max owns:
- Design system (tokens: color, spacing, typography, radius, shadow)
- Visual hierarchy and layout rhythm
- Component visual states (default, hover, active, disabled, error)
- Mobile-first responsive breakpoints
- Accessibility (contrast ratios, ARIA, focus management)

@react-patterns owns:
- Component architecture and file structure
- Hook design and state management
- Performance (memoization, lazy loading, code splitting)
- Composability and reusability patterns
...

3.2 – Extended Frontend Stack for Full Pages

For any complete page (not just a component), extend the core duo into a full stack:

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @react-best-practices
+ @tailwind-patterns + @nextjs-best-practices together.

```



Role breakdown:

- @ui-ux-pro-max: Design system, tokens, visual language
  - @frontend-design: Aesthetics, layout composition, colour usage
  - @react-patterns: Component structure and React best practices
  - @react-best-practices: Performance optimization and React 19+ patterns
  - @tailwind-patterns: Utility-first CSS with Tailwind v4
  - @nextjs-best-practices: App Router, RSC, Server Actions, route handlers
- ...

### ### 3.3 – Conversion & Marketing Pages (Full 6-Skill Stack)

Landing pages, pricing pages, and any conversion-critical surface require all six:

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @tailwind-patterns  
+ @copywriting + @seo-audit + @form-cro together.

- @copywriting: All user-facing text – headlines, CTAs, microcopy, value props
  - @seo-audit: Meta structure, semantic HTML, structured data, page titles
  - @form-cro: Form design optimized for conversion and reduced friction
- ...

### ### 3.4 – Immersive / Animated Design

For scroll experiences, 3D elements, or interactive portfolios:

...

Use @ui-ux-pro-max + @scroll-experience + @3d-web-experience + @react-patterns  
together.

- @scroll-experience: Scroll-driven animations and parallax patterns
  - @3d-web-experience: Three.js and React Three Fiber integration
- ...

### ### 3.5 – Mobile Design

...

Use @ui-ux-pro-max + @mobile-design + @react-patterns together for all mobile-first  
work.

@mobile-design: Touch targets, gesture handling, viewport scaling, iOS/Android quirks.

...

### ### 3.6 – Design Tokens (Mandatory Before Any Styled Component)

Before writing a single styled component, establish:

```
``typescript
// tokens.ts – define FIRST, use everywhere
const tokens = {
 colors: { primary, secondary, accent, surface, error, success },
 spacing: { xs: 4, sm: 8, md: 16, lg: 24, xl: 32, xxl: 48 },
 typography: { fontFamily, fontSizes, fontWeights, lineHeights },
 radii: { sm: 4, md: 8, lg: 16, full: 9999 },
 shadows: { sm, md, lg, xl }
}
...

```

### ### 3.7 – Accessibility Standards (Non-Negotiable)

Every interactive element MUST have:

- `aria-label` or visible label text
- Keyboard navigability (`tabIndex`, `focus-visible` styles)
- Color contrast ratio  $\geq 4.5:1$  (WCAG AA minimum)
- Screen reader compatible semantic markup

### ### 3.8 – Responsive Breakpoints (Test All Before Marking Done)

- `sm` 640px – mobile
- `md` 768px – tablet
- `lg` 1024px – laptop
- `xl` 1280px – desktop
- `2xl` 1536px – wide monitor

---

### ## PHASE 4 – SENIOR CODE DEVELOPMENT {#phase-4}

> **\*\*Trigger:\*\*** Anytime code is being written – features, fixes, refactors, automation, scripts.

### ### 4.1 – Pre-Code Mandatory Checklist

Before writing ANY code, confirm all are true:

- [ ] Architecture decisions are clear and in brain files
- [ ] Types/interfaces are defined before implementation (TypeScript) or type hints written (Python)
- [ ] The function/component has a single, clear responsibility
- [ ] Edge cases are identified and will be handled explicitly
- [ ] The active skill stack covers every discipline this code touches
- [ ] The code can be tested in isolation

### ### 4.2 – Full-Stack Feature Development – The 7-Layer Stack

When building a complete feature end-to-end, all seven layers must be staffed:

...

Layer 1 – Planning:

Use @brainstorming + @concise-planning to spec the feature before touching code.

Layer 2 – Design:

Use @ui-ux-pro-max + @frontend-design + @react-patterns for all UI surfaces.

Layer 3 – Frontend:

Use @react-best-practices + @tailwind-patterns + @nextjs-best-practices.

Layer 4 – Backend:

Use @senior-fullstack + @backend-dev-guidelines + @api-patterns.

Layer 5 – Data:

Use @database-design + @postgres-best-practices.

Layer 6 – Security:

Use @api-security-best-practices + @backend-security-coder +

@frontend-security-coder.

(Security is ALWAYS active – it does not get deactivated for "quick" features.)

Layer 7 – Testing:

Use @test-driven-development + @e2e-testing-patterns.

...

### ### 4.3 – Language & Framework Tandems

**\*\*TypeScript / React / Next.js:\*\***

...

Use @typescript-expert + @react-best-practices + @react-patterns

```

 + @nextjs-best-practices + @nextjs-app-router-patterns together.
 ...

Python / FastAPI:
 ...

Use @python-pro + @python-patterns + @fastapi-pro
 + @async-python-patterns + @python-testing-patterns together.
 ...

Node.js API:
 ...

Use @senior-fullstack + @backend-dev-guidelines + @typescript-expert
 + @nodejs-best-practices + @api-security-best-practices together.
 ...

AI Agent / LLM System:
 ...

Use @ai-agents-architect + @prompt-engineering + @rag-engineer
 + @langgraph + @llm-app-patterns + @context-window-management together.
All six are required for any non-trivial AI system.
 ...

N8N / Make / Automation Workflow:
 ...

Use @workflow-automation (if available) + @api-patterns
 + @api-security-best-practices + @systematic-debugging together.
 ...

Game Development (Unity/Unreal):
 ...

Use @unity-developer + @game-design + @3d-games together for 3D.
Use @unity-developer + @game-design + @2d-games together for 2D.
Also invoke @algorithmic-art for procedural asset generation.
 ...

```

### ### 4.4 – Code Quality Standards (Hard Rules)

| Standard              | Requirement                                                         |
|-----------------------|---------------------------------------------------------------------|
| Function length       | Maximum 40 lines per function                                       |
| Cyclomatic complexity | Maximum 10                                                          |
| Naming                | Self-documenting – no single-letter variables except loop iterators |

```
Comments	Only explain "why", never "what" – the code explains what
Error handling	Every async function has try/catch or an error boundary
Logging	Structured logs with context – no raw `console.log` in production
Secrets	ALL keys in `.env` – never hardcoded, never committed, never logged
```

### ### 4.5 – API Integration Standards

```
...
```

```
Use @api-security-best-practices + @auth-implementation-patterns together
for every external API integration.
```

```
...
```

- API keys: `.env` always – if it's hardcoded, it's wrong
- Webhook payloads: validate signature before processing
- External calls: retry with exponential backoff
- Rate limits: respect provider limits, implement client-side throttling

### ### 4.6 – Stripe / Payments

```
...
```

```
Use @stripe-integration + @api-security-best-practices + @pci-compliance together.
No payment integration is complete without all three.
```

```
...
```

### ### 4.7 – Communications (Twilio, SMS, Voice)

```
...
```

```
Use @twilio-communications + @api-security-best-practices together.
```

```
...
```

```

```

## ## PHASE 5 – SENIOR CODE ANALYSIS & REVIEW {#phase-5}

```
> **Trigger:** Code review, PR review, debugging session, refactoring, or taking over
an existing codebase.
```

### ### 5.1 – Full Code Audit – Mandatory 5-Skill Tandem

```
...
```

```
Use @senior-architect + @lint-and-validate + @vulnerability-scanner
+ @code-review-checklist + @systematic-debugging together.
```

Each skill produces its own report. Reports are then merged:

- @senior-architect: Architectural quality, technical debt, scaling risks
  - @lint-and-validate: Static analysis, code style, type errors
  - @vulnerability-scanner: OWASP Top 10, dependency CVEs, injection risks
  - @code-review-checklist: PR-level checklist, logic errors, missed edge cases
  - @systematic-debugging: Root cause analysis for any identified failures
- ...

### ### 5.2 – Performance Analysis

...

Use @performance-engineer + @react-best-practices together to identify:

...

- N+1 database queries
- Blocking main thread operations
- Unnecessary re-renders (React DevTools Profiler)
- Unoptimized bundle size (``next build && next analyze``)
- Missing image optimization

**\*\*Lighthouse targets:\*\*** ≥ 90 for Performance, Accessibility, Best Practices, SEO

### ### 5.3 – Technical Debt Classification

Label all findings before handing off:

| Label       | Definition                                        | Required Action         |
|-------------|---------------------------------------------------|-------------------------|
| --- --- --- |                                                   |                         |
| 🔴 CRITICAL  | Security vulnerability or data loss risk          | Block merge – fix first |
| 🟡 HIGH      | Broken user flow or major performance issue       | Fix in current sprint   |
| 🟡 MEDIUM    | Code smell, missing test, or anti-pattern         | Fix in next sprint      |
| 🟢 LOW       | Style inconsistency or minor refactor opportunity | Backlog                 |

### ### 5.4 – Dependency Audit

```bash

npm audit --audit-level=high

Python:

pip-audit

...

```

`high` or `critical` severity = **merge blocked until resolved**.

---

## PHASE 6 – SECURITY HARDENING {#phase-6}

> **Trigger:** Before ANY production deployment. After any auth change. Weekly on all
live products.

### 6.1 – The Security Trio (Always All Three, Never Fewer)

...

Use @api-security-best-practices + @backend-security-coder + @frontend-security-coder
together.

These three are NEVER invoked individually for production security work.
They always run as a trio – API layer, server layer, and client layer simultaneously.

For deep audits, extend to the full Security Engineer stack:
Use @ethical-hacking-methodology + @top-web-vulnerabilities + @security-auditor
    + @vulnerability-scanner + @api-security-best-practices together.
...

### 6.2 – Authentication & Authorization Rules

...

Use @auth-implementation-patterns for all auth implementation.
...

- Auth is NEVER built from scratch – use Supabase Auth, NextAuth, Clerk, or Auth0
- Every route has explicit RBAC defined before any code is written
- JWT tokens: 15-minute access, 7-day refresh, never localStorage
- All sensitive operations require explicit re-authentication

### 6.3 – Data Protection

- PII is never logged, anywhere, ever
- PII fields are encrypted at rest
- All transmission uses HTTPS (TLS 1.3 minimum)
- File uploads: validate type AND magic bytes (not just file extension)
- ALL SQL queries: parameterized statements only – no string interpolation

### 6.4 – Mandatory Security Headers

```

```
```http
Content-Security-Policy: default-src 'self'; script-src 'self'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
Referrer-Policy: no-referrer-when-downgrade
Permissions-Policy: camera=(), microphone=(), geolocation=()
```
```

6.5 – Payments Security

```
```
Use @pci-compliance + @stripe-integration + @api-security-best-practices together.
Never handle raw card data – always use Stripe Elements or Stripe Checkout.
```
```

PHASE 7 – INFRASTRUCTURE & DEPLOYMENT {#phase-7}

```
> **Trigger:** Before any deployment – staging or production.
```

7.1 – Deployment Tandem

```
```
Use @deployment-procedures + @docker-expert + @observability-engineer together
for all deployments.
- @deployment-procedures: safe rollout strategy, canary/blue-green, rollback plan
- @docker-expert: container build optimization, multi-stage Dockerfiles,
compose
- @observability-engineer: monitoring setup, alerting thresholds, health checks
For AWS: add @aws-serverless as a fourth skill.
For K8s: add @kubernetes-architect as a fourth skill.
For IaC: add @terraform-specialist as a fifth skill.
```
```

7.2 – Environment Management

Every project runs three environments with no exceptions:

| Environment | Purpose | Auto-Deploy | Approval Required |
|-------------|---------|-------------|-------------------|
|-------------|---------|-------------|-------------------|


```
|---|---|---|---|
```

```
`development`	Local dev, experiments	No	No
`staging`	Pre-prod testing, client demos	Yes (on PR)	No
`production`	Live user traffic	No	Yes – manual only
```

7.3 – Pre-Deployment Checklist

- [] All environment variables set in deployment platform (NOT in `.env` file committed to git)
- [] Database migrations are backwards-compatible (can roll back without data loss)
- [] Health check endpoint active and returns 200 (`/api/health`)
- [] Error tracking configured (Sentry or equivalent)
- [] Rollback procedure documented AND tested
- [] Rate limiting active on all public-facing endpoints
- [] Security headers verified

7.4 – CI/CD Pipeline – Required Order

1. `lint` – fail fast on style and formatting
2. `type-check` – TypeScript `tsc --noEmit` or mypy
3. `test` – unit + integration
4. `build` – production build must succeed
5. `security-scan` – `npm audit --audit-level=high`
6. `deploy` – only on `main` or tagged release branch, after human approval for production

```
---
```

PHASE 8 – TESTING & QA {#phase-8}

> ****Trigger:**** After every feature. Before every PR. Before every deployment.

8.1 – The QA Skill Stack (All Five Active Together)

```
...
```

Use @test-driven-development + @systematic-debugging + @browser-automation
+ @e2e-testing-patterns + @test-fixing together.

Role breakdown:

- @test-driven-development: Red/green/refactor cycle – write tests first
- @systematic-debugging: Root cause analysis when tests fail
- @browser-automation: Playwright-based E2E and browser automation

```

- @e2e-testing-patterns:      Reliable, non-flaky E2E test suite design
- @test-fixing:               Systematic approach to fixing broken tests after
refactors
...

### 8.2 - Test Coverage Requirements

Layer	Minimum Coverage	Test Type
Pure utility functions	100%	Unit
API routes / handlers	90%	Integration
Critical UI components	80%	Component
Critical user journeys	100%	E2E

### 8.3 - Non-Negotiable E2E Tests (Must Exist Before Production)

- User registration and email verification
- Login, logout, session expiry
- Core conversion action (signup CTA, purchase, booking, form submission)
- Payment flow (if applicable - full Stripe test mode run)
- Data submission and retrieval round-trip
- Error states (network failure, validation errors, 404, 500)
- Empty states (new user, no data, no results)

### 8.4 - Performance Benchmarks

```bash
npx lhci autorun
Load test:
npx autocannon -c 100 -d 30 https://staging.yourapp.com/api/health
...

PHASE 9 - DOCUMENTATION & HANDOFF {#phase-9}

> **Trigger:** After every major feature. At project completion. Before any handoff.

9.1 - Documentation Tandem

...

Use @documentation-templates + @changelog-automation together.

```

```

@documentation-templates: standardized docs structure and handoff format.
@changelog-automation: keep release notes consistent and up to date automatically.
...

9.2 – Mandatory Documentation Set

| Document | Location | Owner Skill | Contents |
|---|---|---|---|
| `README.md` | Root | `@documentation-templates` | Setup, run, deploy, env vars |
| `ARCHITECTURE.md` | `/docs` | `@architecture-decision-records` | System design,
ADRs, data flow |
| `API.md` | `/docs` | `@api-patterns` | All endpoints, payloads, auth, error codes |
| `ENVIRONMENT.md` | `/docs` | `@deployment-procedures` | All env vars with
descriptions and examples |
| `CHANGELOG.md` | Root | `@changelog-automation` | Version history and migration
notes |
| `brain/` | `.gemini/antigravity/brain/` | Agent memory | Persistent decisions, skill
tandems, session logs |

9.3 – Code-Level Documentation Standards

- Every exported function: JSDoc comment or Python docstring
- Every API endpoint: OpenAPI/Swagger annotation
- Every complex algorithm: plain-English explanation comment above it
- Every `TODO`: includes `@owner-name` and a GitHub issue number

PHASE 10 – POST-DEPLOYMENT & OBSERVABILITY {#phase-10}

> Trigger: Immediately after production deployment. Ongoing weekly cadence.

10.1 – Observability Stack (Minimum Viable Production)

...

Use @observability-engineer + @distributed-tracing + @slo-implementation together
for production monitoring setup.

- @observability-engineer: full monitoring stack, alerting, dashboards
- @distributed-tracing: request tracing across microservices/edge functions
- @slo-implementation: Service Level Objectives and error budget tracking
...

```

### ### 10.2 – Required Monitoring Stack

- **\*\*Error Tracking:\*\*** Sentry (free tier covers most projects)
- **\*\*Uptime Monitoring:\*\*** UptimeRobot or Better Uptime
- **\*\*Performance:\*\*** Vercel Analytics or Plausible
- **\*\*Log Aggregation:\*\*** Axiom, Logtail, or Vercel Logs

### ### 10.3 – Weekly Maintenance Skills

...

Mondays: Use @vulnerability-scanner to run weekly dependency and security scan.

Monthly: Use @seo-audit to run SEO health check on all client-facing properties.

Monthly: Use @performance-engineer to run performance regression check.

...

### ### 10.4 – Alerting Thresholds

Metric	Warning Threshold	Critical Threshold
Error rate	>1%	>5%
P99 API latency	>2s	>5s
Uptime	<99.9%	<99%
Failed deployments	Any	Any
Security scan findings	Any HIGH	Any CRITICAL

---

### ## ⚡ MASTER TANDEM PLAYBOOK – PRE-BUILT COMBOS BY PROJECT TYPE {#tandem-playbook}

> Copy any of these directly into your agent prompt. These are the correct skill combinations for each common project type.

### ### 🌐 Landing Page / Marketing Site

...

Use @ui-ux-pro-max + @frontend-design + @react-patterns + @tailwind-patterns  
+ @copywriting + @seo-audit + @form-cro together.

...

### ### 📊 SaaS Dashboard / Admin Panel

...

Use @ui-ux-pro-max + @react-best-practices + @react-patterns + @nextjs-best-practices  
+ @senior-fullstack + @database-design + @postgres-best-practices

```
+ @api-security-best-practices together.
...

🗝️ Authentication System
...

Use @auth-implementation-patterns + @backend-security-coder + @frontend-security-coder
 + @typescript-expert + @test-driven-development together.
...

🗣️ AI Chatbot / Agent System
...

Use @ai-agents-architect + @prompt-engineering + @rag-engineer + @langgraph
 + @llm-app-patterns + @context-window-management + @langfuse together.
...

🇪🇺 E-Commerce / Payment Flow
...

Use @stripe-integration + @pci-compliance + @api-security-best-practices
 + @ui-ux-pro-max + @copywriting + @form-cro together.
...

⚙️ N8N / Automation Workflow Build
...

Use @api-patterns + @api-security-best-practices + @systematic-debugging
 + @backend-dev-guidelines together.
...

🎮 Game Development (Unity / Unreal)
...

For 3D: Use @unity-developer + @game-design + @3d-games + @algorithmic-art together.
For 2D: Use @unity-developer + @game-design + @2d-games + @algorithmic-art together.
...

📱 Mobile App (React Native / Flutter)
...

Use @mobile-developer + @react-native-architecture + @mobile-design
 + @api-security-best-practices + @app-store-optimization together.
...

🔍 Full Codebase Audit
...

Use @senior-architect + @vulnerability-scanner + @lint-and-validate
```

```

 + @code-review-checklist + @security-auditor + @performance-engineer together.
 ...

🚀 Full SaaS MVP (Ship in 2 Weeks)
 ...

Week 1 - Build:
 Use @brainstorming + @concise-planning + @product-manager-toolkit (Day 1: spec)
 Use @ui-ux-pro-max + @react-patterns + @tailwind-patterns + @nextjs-best-practices
 (Days 2-4: frontend)
 Use @senior-fullstack + @database-design + @api-security-best-practices (Days 2-4:
 backend)

Week 2 - Ship:
 Use @test-driven-development + @e2e-testing-patterns + @browser-automation (Days 8-9:
 tests)
 Use @api-security-best-practices + @backend-security-coder (Day 10: security
 hardening)
 Use @deployment-procedures + @docker-expert + @observability-engineer (Days 11-14:
 deploy)
 ...

🛡️ Security Audit & Hardening
 ...

Use @ethical-hacking-methodology + @top-web-vulnerabilities + @vulnerability-scanner
 + @api-security-best-practices + @backend-security-coder + @frontend-security-coder
 + @security-auditor together.
 ...

📈 Growth & Conversion Optimization
 ...

Use @seo-audit + @programmatic-seo + @analytics-tracking + @ab-test-setup
 + @copywriting + @form-cro + @content-creator together.
 ...

🏢 Enterprise / DDD Architecture
 ...

Use @domain-driven-design + @ddd-strategic-design + @ddd-tactical-patterns
 + @cqrs-implementation + @event-sourcing-architect + @microservices-patterns
 together.
 ...

```

## 

### ### Invocation Syntax Quick Reference

Tool	Single Skill	Multi-Skill Tandem
------	--------------	--------------------

— — — — —

11

### 🚀

111

```
@concise-planning - Always start with a plan
```

```
@lint-and-validate - Keep code clean automatically
```

```
@systematic-debugging - Debug like a professional
```

111

### 

111

```
@frontend-design - The base layer of UI aesthetics
```

@3d-web-experience - Three.js & React Three Fiber

```
@canvas-design - Static visuals and posters
```

```
@scroll-experience - Immersive scroll-driven experiences
```

111

### 

5

```

@frontend-design - UI guidelines and aesthetics
@react-best-practices - React & Next.js performance optimization
@react-patterns - Modern React patterns and principles
@nextjs-best-practices - Next.js App Router patterns
@tailwind-patterns - Tailwind CSS v4 styling
@form-cro - Forms optimized for conversion
@seo-audit - Technical SEO health checks

```

### ### ⚡ Full-Stack Developer Pack

```

@senior-fullstack - Complete full-stack development guide
@frontend-developer - React 19+ and Next.js 15+ expertise
@backend-dev-guidelines - Node.js/Express/TypeScript patterns
@api-patterns - REST vs GraphQL vs tRPC selection
@database-design - Schema design and ORM selection
@stripe-integration - Payments and subscriptions

```

### ### 🤖 Agent Architect Pack

```

@agent-evaluation - Test and benchmark your agents
@langgraph - Stateful agent workflows
@mcp-builder - Create custom MCP tools
@prompt-engineering - LLM prompt design and optimization
@ai-agents-architect - Design autonomous AI agents
@rag-engineer - RAG systems with vector search

```

### ### 🧠 LLM Application Developer Pack

```

@llm-app-patterns - Production-ready LLM patterns
@rag-implementation - Retrieval-Augmented Generation
@prompt-caching - Cache strategies for LLM prompts
@context-window-management - Manage LLM context efficiently
@langfuse - LLM observability and tracing

```

### ### 🐍 Python Pro Pack

```

@python-pro - Python 3.12+ modern features

```



```
@python-patterns - Idiomatic Python code
@fastapi-pro - High-performance async APIs
@fastapi-templates - Production-ready FastAPI
@async-python-patterns - Python asyncio mastery
@python-testing-patterns - pytest comprehensive testing
...
```

### ### TypeScript & JavaScript Pack

```
...
@typescript-expert - TypeScript mastery, strict mode, generics
@javascript-pro - Modern JavaScript with ES6+
@react-best-practices - React performance optimization
@nodejs-best-practices - Node.js development principles
@nextjs-app-router-patterns - Next.js 14+ App Router
...
```

### ### Security Engineer Pack

```
...
@ethical-hacking-methodology - The methodology for ethical hacking
@top-web-vulnerabilities - OWASP-aligned vulnerability taxonomy
@security-auditor - Comprehensive security audits
@vulnerability-scanner - Advanced vulnerability analysis
@cloud-penetration-testing - AWS/Azure/GCP security
...
```

### ### Security Developer Pack

```
...
@api-security-best-practices - Secure API design patterns
@auth-implementation-patterns - JWT, OAuth2, session management
@backend-security-coder - Secure backend coding
@frontend-security-coder - XSS prevention, client-side security
@cc-skill-security-review - Security checklist for features
@pci-compliance - Payment card security standards
...
```

### ### QA & Testing Pack

```
...
@test-driven-development - Red, Green, Refactor cycle
@systematic-debugging - Debug like Sherlock Holmes
@browser-automation - E2E testing with Playwright
@e2e-testing-patterns - Reliable E2E test suites
@code-review-checklist - Catch bugs in PRs
```

```
@test-fixing - Fix failing tests systematically
...

```

### ### ☁️ DevOps & Cloud Pack

```
...
@docker-expert - Containers and multi-stage builds
@aws-serverless - Lambda, DynamoDB, serverless
@kubernetes-architect - K8s architecture and GitOps
@terraform-specialist - Infrastructure as Code
@deployment-procedures - Safe rollout strategies
@bash-linux - Terminal wizardry
...

```

### ### 📊 Observability & Monitoring Pack

```
...
@observability-engineer - Comprehensive monitoring systems
@distributed-tracing - Track requests across microservices
@slo-implementation - Service Level Objectives
@incident-responder - Rapid incident response
@postmortem-writing - Blameless postmortems
@performance-engineer - Application performance optimization
...

```

### ### 🦄 Startup Founder Pack

```
...
@product-manager-toolkit - RICE prioritization, PRD templates
@competitive-landscape - Competitor analysis
@launch-strategy - Product launch planning
@copywriting - Marketing copy that converts
@stripe-integration - Get paid from day one
...

```

### ### 📈 Marketing & Growth Pack

```
...
@content-creator - SEO-optimized marketing content
@seo-audit - Technical SEO health checks
@programmatic-seo - Pages at scale
@analytics-tracking - GA4/PostHog setup
@ab-test-setup - Validated learning experiments
@email-sequence - Automated email campaigns
...

```

### ### 🎮 Game Dev Pack

...

@game-design	- Mechanics and game loops
@2d-games	- Sprites and 2D physics
@3d-games	- 3D models and shaders
@unity-developer	- Unity 6 LTS development
@godot-gdscript-patterns	- Godot 4 GDScript patterns
@algorithmic-art	- Generate assets with code

...

### ### 🏗️ Architecture & Design Pack

...

@senior-architect	- Comprehensive software architecture
@architecture-patterns	- Clean Architecture, DDD, Hexagonal
@microservices-patterns	- Microservices architecture
@event-sourcing-architect	- Event sourcing and CQRS
@architecture-decision-records	- Document technical decisions

...

### ### 🧱 DDD & Evented Architecture Pack

...

@domain-driven-design	- Strategic to tactical DDD routing
@ddd-strategic-design	- Bounded contexts, ubiquitous language
@ddd-tactical-patterns	- Aggregates, value objects, repositories
@cqrs-implementation	- Read/write model separation
@event-store-design	- Event persistence and replay
@saga-orchestration	- Cross-context transaction coordination
@projection-patterns	- Materialized read models from events

...

### ### 🛠️ OSS Maintainer Pack

...

@commit	- High-quality conventional commits
@create-pr	- PR creation with review-ready context
@changelog-automation	- Consistent release notes
@git-advanced-workflows	- Rebase, cherry-pick, bisect, recovery
@documentation-templates	- Standardize docs and handoffs

...

### ### 🔗 Integrations Pack

...

@stripe-integration	- Payments and subscriptions
---------------------	------------------------------

```

@twilio-communications - SMS, voice, WhatsApp
@hubspot-integration - CRM integration
@algolia-search - Search implementation
@analytics-tracking - GA4/PostHog setup
...

🚨 EMERGENCY PROTOCOLS {#emergency}

Production Is Down

...

1. DO NOT make changes without a documented rollback plan.
2. Use @observability-engineer + @systematic-debugging + @incident-responder together
 to diagnose the failure and coordinate response.
3. Check logs FIRST — never guess the cause.
4. Roll back to last known good deployment.
5. Use @postmortem-writing to write a blameless post-mortem within 24 hours.
...

Security Breach Suspected

...

1. IMMEDIATELY revoke all active API keys, sessions, and tokens.
2. Use @vulnerability-scanner + @api-security-best-practices
 + @ethical-hacking-methodology together to assess the full attack surface.
3. Do not discuss the breach publicly until investigation is complete.
4. Notify affected users within 72 hours (GDPR/compliance requirement).
5. Use @security-auditor to document findings and remediation.
...

Runaway API Costs (Google AI Studio / Gemini)

...

1. Immediately add hard spend cap in Google Cloud Console.
2. Use @api-security-best-practices + @observability-engineer together
 to identify the source of runaway calls and add rate limiting.
3. Enable billing alerts at 50%, 80%, and 100% of budget threshold.
4. Review all public endpoints for missing authentication.
...

```

### ### Critical Bug in Production

```
...
Use @systematic-debugging + @incident-responder together for active debugging.
Use @test-driven-development to write a regression test before deploying the fix.
Never deploy a fix without a test that catches it.
...
```

---

## ## 🧠 PROJECT BRAIN MANAGEMENT {#brain}

### ### Brain Directory Structure

```
...
.gemini/antigravity/brain/
├─ architecture.md # System design decisions and ADRs
├─ stack.md # Tech stack with justifications and alternatives rejected
├─ decisions.md # Key decisions with rationale, date, and owner
├─ conventions.md # Naming, folder structure, code conventions
├─ integrations.md # All external APIs, key structure, webhook endpoints
├─ known-issues.md # Known bugs, technical debt, workarounds
├─ skill-tandems.md # ★ Successful skill combos discovered in this project
└─ session-log.md # Summary of what was accomplished each session
...
```

> `skill-tandems.md` is critical. Every time a skill combination works exceptionally well for this specific project, log it here so every future agent session can build on it without rediscovering it.

### ### Brain Read Protocol (Every Session Start)

```
```bash  
# Run this before anything else in every session  
for f in architecture stack decisions conventions integrations skill-tandems; do  
  echo "=== $f ===" && cat ".gemini/antigravity/brain/$f.md" 2>/dev/null || echo "(not  
yet written)"  
done  
...
```

Brain Write Protocol (Every Session End and After Major Decisions)

Write to brain after:

- Every architectural decision (with rationale and date)
- Every new convention established
- Every new external integration added
- Every successful new skill tandem discovered
- Every significant bug found AND its root cause
- Every session end (what was accomplished, what is next)

 SESSION COMPLETION CHECKLIST {#checklist}

Before ending ANY work session, every item must be checked:

****Code Quality****

- [] Lint and type checks pass (`npm run lint && tsc --noEmit`)
- [] No secrets or API keys anywhere in committed code
- [] All new code has corresponding tests
- [] No `console.log` statements in production paths

****Documentation****

- [] Brain files updated with session summary and any new decisions
- [] `brain/skill-tandems.md` updated with effective skill combinations used today
- [] CHANGELOG.md updated with what changed
- [] Outstanding TODOs have GitHub issue numbers assigned

****Deployment (if applicable)****

- [] Staging deployment verified and tested
- [] Security headers confirmed active
- [] Monitoring alerts confirmed active
- [] Rollback procedure confirmed available

****Handoff****

- [] Next session's starting point is documented in `brain/session-log.md`
- [] Any blockers or open questions are written down explicitly

 RAPIDXAI PROJECT TYPE QUICK-SELECT

For RapidXAI client work specifically, use these pre-selected stacks:

Client Type	Lead Skills	Tandem	Support Skills
HVAC / Plumbing / Solar	`@ai-agents-architect + @prompt-engineering`		`@ui-ux-pro-max + @senior-fullstack + @stripe-integration`
Real Estate Agency	`@rag-engineer + @langgraph`		`@react-best-practices + @seo-audit + @copywriting`
E-Commerce / Shopify	`@stripe-integration + @frontend-developer`		`@ui-ux-pro-max + @form-cro + @seo-audit`
Dental / Hair Salon	`@ai-agents-architect + @twilio-communications`		`@ui-ux-pro-max + @copywriting + @seo-audit`
Tour Operators	`@senior-fullstack + @database-design`		`@ui-ux-pro-max + @react-patterns + @stripe-integration`
SaaS Product (RapidXAI)	`@senior-fullstack + @ai-agents-architect`		`@react-best-practices + @api-security-best-practices + @product-manager-toolkit`

This SOP is a living document. Every agent that reads it is expected to follow it with zero deviation unless explicitly overridden by the project owner. It is updated as the project evolves, new skills are discovered, and better tandem combinations are found.

Built for RapidXAI. Powered by Antigravity Awesome Skills (868+ skills – github.com/sickn33/antigravity-awesome-skills). February 2026.

Tab 2

Website Copy Prompt:

Here is your complete master prompt. It's engineered as a two-stage system — the **Extractor Prompt** you feed into Gemini with your screenshot/video, and the **structured output format** it produces, which you paste directly into Google Antigravity or Claude Code.

The Master Website Aesthetic Extractor Prompt

Copy everything below the divider and paste it into Gemini with your screenshot or screen recording attached:

text

You are an expert UI/UX Design Analyst and Senior Frontend Engineer. Your task is to perform a **complete aesthetic forensic analysis** of the attached website screenshot or video.

Your goal: Produce a **structured, copy-paste-ready Design Implementation Sheet** – a single output document so precise and complete that any developer (or AI coding agent like Google Antigravity, Claude Code, or GPT-4o) can rebuild this website's exact visual style from scratch, without ever seeing the original.

ANALYSIS INSTRUCTIONS

Examine the attached image or video frame-by-frame and extract ALL of the following. Be precise. Use exact hex codes, exact pixel values, exact font names, exact CSS values wherever possible. Where you cannot determine the exact value, give the closest professional estimate and flag it with `[ESTIMATED]`.

Do NOT describe what the website is about. ONLY extract the visual and design system.

OUTPUT FORMAT

Your output must follow this exact structure. Do not skip any section.

SECTION 1 - DESIGN PERSONALITY BRIEF

Write 3-5 sentences describing the overall visual "vibe" and design language. Include:

- The emotional mood (e.g. dark and luxurious, clean and minimal, bold and energetic)
- Design style category (e.g. glassmorphism, brutalism, neo-brutalism, soft UI, editorial, corporate clean, streetwear dark)
- Target audience aesthetic inference
- Overall design maturity level (e.g. polished SaaS, indie hacker, agency premium)

SECTION 2 - COLOR SYSTEM

List every color visible in the design with:

- Role (e.g. Background Primary, Surface Card, Text Primary, Text Muted, Accent Primary, Accent Secondary, Border, CTA Button, Hover State, Error, Success)
- Hex code
- RGB equivalent
- Where it is used

Then write the complete CSS custom properties block:

```
```css
:root {
 --color-bg-primary: #_____;
 --color-bg-secondary: #_____;
 --color-surface: #_____;
 --color-text-primary: #_____;
 --color-text-secondary: #_____;
 --color-text-muted: #_____;
}
```

```
--color-accent-1: #_____;
--color-accent-2: #_____;
--color-border: #_____;
--color-cta: #_____;
--color-cta-hover: #_____;
}
```

Also specify if the site uses a dark mode, light mode, or both.

---

## SECTION 3 — TYPOGRAPHY SYSTEM

For each font used, provide:

- Font family name (attempt to identify the exact font — Google Font, Adobe, custom, etc.)
- Fallback stack
- Font weights used
- All type scale sizes used (px and rem)
- Line height values
- Letter spacing values
- Text transform usage (uppercase, lowercase, none)
- Where each font/style is used (headings, body, labels, CTAs, etc.)

Then write the complete CSS typography block:

CSS

```
/* TYPOGRAPHY SYSTEM */
--font-heading: 'FontName', fallback, sans-serif;
--font-body: 'FontName', fallback, sans-serif;
--font-mono: 'FontName', monospace;

--text-xs: 0.75rem; /* 12px */
--text-sm: 0.875rem; /* 14px */
--text-base: 1rem; /* 16px */
--text-lg: 1.125rem; /* 18px */
--text-xl: 1.25rem; /* 20px */
--text-2xl: 1.5rem; /* 24px */
--text-3xl: 2rem; /* 32px */
--text-4xl: 2.5rem; /* 40px */
```

```
--text-5xl: 3.5rem; /* 56px */

--font-weight-regular: 400;
--font-weight-medium: 500;
--font-weight-semibold: 600;
--font-weight-bold: 700;
--font-weight-black: 900;

--leading-tight: 1.1;
--leading-snug: 1.25;
--leading-normal: 1.5;
--leading-relaxed: 1.75;

--tracking-tight: -0.03em;
--tracking-normal: 0em;
--tracking-wide: 0.05em;
--tracking-widest: 0.15em;
```

---

## SECTION 4 — SPACING & LAYOUT SYSTEM

- Base spacing unit (4px, 8px, etc.)
- Container max-width
- Section padding (top/bottom)
- Content padding (left/right)
- Grid system: number of columns, gap size
- Card padding
- Button padding

Write the CSS spacing block:

css

```
--spacing-unit: 8px;
--spacing-xs: 4px;
--spacing-sm: 8px;
--spacing-md: 16px;
--spacing-lg: 24px;
--spacing-xl: 32px;
--spacing-2xl: 48px;
```

```
--spacing-3xl: 64px;
--spacing-4xl: 96px;

--container-max-width: _____px;
--section-padding-y: _____px;
--content-padding-x: _____px;
--grid-columns: _____;
--grid-gap: _____px;
```

---

## SECTION 5 — BORDER & SHAPE SYSTEM

- Border radius values used (none, slight, medium, pill, full circle)
- Border widths used
- Border styles (solid, dashed, gradient border)
- Border colors
- Which elements have borders (cards, inputs, buttons, sections, images)

css

```
--radius-sm: _____px;
--radius-md: _____px;
--radius-lg: _____px;
--radius-xl: _____px;
--radius-full: 9999px;

--border-width: _____px;
--border-color: #_____
--border-style: solid;
```

---

## SECTION 6 — SHADOW & DEPTH SYSTEM

For every shadow style visible:

- Type (box-shadow, drop shadow, inner shadow, glow)
- Target element
- Exact CSS value (or closest estimate)
- Color of shadow (note if colored glow shadows are used)

CSS

```
--shadow-sm: 0 1px 3px rgba(0,0,0,0.____);
--shadow-md: 0 4px 16px rgba(0,0,0,0.____);
--shadow-lg: 0 8px 32px rgba(0,0,0,0.____);
--shadow-glow-accent: 0 0 24px rgba(____,____,____,0.____);
```

---

## SECTION 7 — VISUAL EFFECTS & TEXTURE

Identify and describe:

- Glassmorphism: backdrop-filter blur levels, background opacity
- Gradients: type (linear/radial/conic), direction, exact color stops
- Noise/grain texture: yes/no, intensity
- Background patterns: grid lines, dots, noise, geometric shapes
- Image overlays or tints
- Frosted glass panels
- Mesh gradients or aurora effects

Write the CSS for each:

CSS

```
/* Example */
--glass-bg: rgba(255,255,255,0.08);
--glass-blur: blur(12px);
--glass-border: 1px solid rgba(255,255,255,0.12);

--gradient-hero: linear-gradient(135deg, #_____ 0%, #_____ 100%);
--gradient-cta: linear-gradient(90deg, #_____ 0%, #_____ 100%);
```

---

## SECTION 8 — COMPONENT BREAKDOWN

For each UI component visible, write a precise description AND the CSS spec:

### Buttons

- Primary CTA button: background, text color, border, border-radius, padding, font-weight, hover state, shadow, any gradient or glow

- Secondary button: same fields
- Ghost/outline button: same fields

## Navigation / Header

- Background style (transparent, frosted, solid, sticky behavior)
- Logo area
- Nav link style (font, size, weight, color, hover state)
- Active state style
- CTA in nav

## Cards / Panels

- Background, border, border-radius, padding, shadow
- Hover effect
- Any inner layout pattern

## Input Fields / Forms

- Background, border, border-radius, font, placeholder color, focus ring color

## Tags / Badges / Pills

- Background, text color, border-radius, font size, padding

## Section Dividers / Separators

- Style used (line, gradient fade, spacing only, decorative shape)

---

## SECTION 9 — ANIMATION & MOTION LANGUAGE

Identify:

- Hover animations (scale, color shift, glow, underline draw, lift)
- Scroll-triggered animations (fade in, slide up, stagger)
- Page transition style
- Loading/skeleton states if visible
- Cursor effects if any
- Micro-interaction patterns (button press, toggle)

Write CSS transition specs:

CSS

```
--transition-fast: 150ms ease;
--transition-base: 250ms ease;
--transition-slow: 400ms cubic-bezier(0.16, 1, 0.3, 1);
--transition-spring: 500ms cubic-bezier(0.34, 1.56, 0.64, 1);
```

---

## SECTION 10 — ICONOGRAPHY & IMAGERY STYLE

- Icon style: line icons, filled, duotone, 3D, custom illustrated, emoji-based
  - Icon size scale
  - Image treatment: photos, illustrations, 3D renders, AI-generated
  - Image border-radius, overlay style, aspect ratios used
  - Any consistent image filter or treatment (e.g. desaturated, duotone tint, warm filter)
- 

## SECTION 11 — SECTION LAYOUT PATTERNS

Describe the layout pattern of each major section visible:

- Hero: layout type, alignment, background treatment, CTA placement
  - Feature section: grid or list, icon/image placement
  - Testimonials: card style, avatar treatment
  - Footer: column count, link style, background
- 

## SECTION 12 — IMPLEMENTATION PROMPT

This is the most critical section. Write a **single, self-contained, copy-paste prompt** that someone can paste directly into Google Antigravity, Claude Code, or any AI coding agent to build a new website that matches the extracted style EXACTLY.

This prompt must:

1. Be written in imperative, direct language ("Build a...", "Use...", "Apply...")
2. Include ALL CSS variables from Sections 2–9 in a single `:root` block
3. Specify the exact font imports (Google Fonts `@import` link or equivalent)
4. Describe the visual language in 2–3 sentences so the AI understands the vibe



5. Instruct the AI to build with HTML + CSS + vanilla JS OR React + Tailwind (specify whichever matches the original)
  6. Include exact component specs for: hero, navbar, cards, buttons, footer
  7. Include animation instructions
  8. End with: "Do not deviate from the design system above. Every element must use the defined CSS variables. The final output must visually match the reference aesthetic."
- 

## OUTPUT RULES:

- Be EXHAUSTIVE. Do not summarize or skip details.
- Use technical language. This output is for developers and AI agents.
- Every color must be a hex code. No "dark blue" or "warm gray."
- Every size must be a number. No "small" or "large."
- If something is not visible or unclear, write: [NOT VISIBLE – RECOMMEND: \_\_\_\_\_] and suggest a matching value.
- Do NOT include explanations of what the website does. Only design data.
- Keep each section tight and structured. Use code blocks for all CSS.
- Total output should be comprehensive but NOT redundant. Say each thing once, precisely.

text

\*\*\*

## How to Use This in Your Workflow

Here's the exact workflow to show in your video:[1][2]

1. **\*\*Attach the screenshot or screen recording\*\*** of any website to Gemini 3 in Google AI Studio (or the Antigravity chat), then paste the entire prompt above
2. **\*\*Gemini runs the analysis\*\*** and produces the full Design Implementation Sheet
3. **\*\*Copy Section 12 (The Implementation Prompt)\*\*** – this is your golden paste-in block
4. **\*\*Open Google Antigravity\*\***, start a new project, and paste Section 12 directly as your first message
5. **\*\*Follow up\*\*** with Section 2–9's CSS variables block to ensure Antigravity locks in the exact tokens

## ## Pro Tips for Your Video

- Google Antigravity's agent can accept the full CSS `:root` block as a follow-up message after the initial build, telling it to `*"replace all hardcoded colors and values with these CSS variables"*`[3]
- For `**output length management**`, if Gemini starts hitting its response limit, simply reply: ``"Continue from SECTION ___"`` – the modular structure of this prompt makes this seamless
- You can also split the analysis into two Gemini calls: Call 1 extracts Sections 1–7 (design tokens), Call 2 extracts Sections 8–12 (components + implementation prompt)[4]
- This prompt works with `**Gemini 3, Claude 3.7, GPT-4o, and Grok 3**`
- all support vision and structured output[5]